



UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DEL NORTE "CUNOR"
COBÁN, GUATEMALA

Curso: Lenguajes formales de programación

Proyecto: Proyecto Final - Analizador Léxico

Entregable 3: Implementación del motor léxico

Docente: Ingeniero José Alberto Veliz Cruz

Integrantes

Nombre	Carné	Carrera
Cristian Gabriel Ramírez Morales	202540362	Sistemas
Enrique José Sosa Caal	202544363	Sistemas
José Roberto Posadas Ascencio	202544488	Sistemas

Enlace de GitHub: <https://github.com/Zenite50/C-Extended>

Fecha de entrega: 28 de septiembre de 2026

Motor léxico en C#

Docente: Ing. José Alberto Veliz Cruz. Integrantes: Cristian Gabriel Ramírez Morales (202540362), Enrique José Sosa Caal (202544363) y José Roberto Posadas Ascencio (202544488). Preparación: 28 de septiembre de 2026.

Alcance de esta etapa

Se implementó el núcleo del analizador a partir del esqueleto del Entregable 2. Procesa archivos de texto y entrega tokens con lexema, categoría, línea y columna; una tabla de símbolos sin duplicados; y errores léxicos con recuperación. El formulario gráfico conservado en la solución corresponde a la integración prevista para el Entregable 4.

Arquitectura

Clase	Responsabilidad
AnalizadorLexico	Recorrido carácter por carácter; estados, posiciones y recuperación.
AutomataAFD	Alfabeto de identificadores, palabras reservadas y escapes.
Token / ErrorLexico	Resultados con lexema o secuencia inválida y ubicación.
TablaDeSimbolos	Identificadores únicos y primera aparición.

Decisiones de implementación

- Las ramas de reconocimiento aplican transiciones manuales; no se utiliza Regex de .NET como motor. Se priorizan operadores de dos caracteres y comentarios antes que división.
- Las posiciones empiezan en 1 y CRLF cuenta como un solo salto de línea. Un nuevo análisis reinicia tokens, errores y símbolos.
- Se conservan las palabras del subconjunto propio (por ejemplo, func, var, nil) y se cubren las categorías mínimas del proyecto general, incluidos caracteres literales y operadores lógicos.

Pruebas y resultados

Se ejecutó un programa de consola con los tres archivos del Entregable 1 y dos archivos nuevos. Además de los conteos, las 28 verificaciones comprueban categorías, posiciones, unicidad de símbolos, reinicio y continuidad tras errores.

Archivo	Tokens	Símbolos	Errores	Resultado
prueba1_sencillo.txt	25	3	0	Aprobado
prueba2_medio.txt	62	9	0	Aprobado
prueba3_errores.txt	20	4	4	Aprobado
prueba4_recuperacion.txt	21	4	2	Aprobado
prueba5_categorias.txt	59	7	0	Aprobado

Errores observados en la prueba original

Ubicación	Secuencia	Diagnóstico
6:13	12variable	Identificador con inicio numérico
9:22	45.12.3	Número decimal mal formado
12:23	Cadena sin cierre	Cadena de texto sin cerrar
14:9	/* ... EOF	Comentario multilínea sin cerrar

Interpretación: el carácter @ del archivo original aparece después de /* sin cierre. Perteneció al texto del comentario y no debe diagnosticarse por separado. En prueba4_recuperacion.txt, situado fuera de un comentario, se reporta en 2:1 y el análisis continúa hasta el identificador final.

Resultado global: 28 verificaciones aprobadas y 0 fallidas. La salida completa se conserva en Pruebas/evidencia_pruebas_entregable3.txt.

Validación y cierre

Dificultades y soluciones

- Esqueleto sin lógica: el ZIP contenía las clases del núcleo vacías. Se implementó el reconocimiento en esas clases y se mantuvo separado del formulario.
- Errores encadenados: un lexema numérico inválido se consume completo para evitar tokens falsos; una cadena sin cerrar termina al fin de línea y permite continuar. Un comentario de bloque sin cierre consume hasta EOF, según su semántica.
- Inconsistencia del archivo de errores: el supuesto quinto error queda dentro del comentario abierto. Se agregó un caso independiente para verificar el carácter prohibido y la recuperación.

Compilación y ejecución reproducible

La solución original de Windows Forms (objetivo .NET Framework 4.8) compiló sin advertencias ni errores en este equipo. La suite de consola se ejecutó con SDK .NET 10 y produjo 28/28 verificaciones aprobadas. Desde la raíz de C Extended:

```
dotnet run --project PruebasConsola\PruebasConsola.csproj -- .\Pruebas
```

El comando de compilación del proyecto antiguo desde el SDK instalado requirió la propiedad `GenerateResourceMSBuildArchitecture=CurrentArchitecture` por la tarea de recursos de MSBuild. En Visual Studio con las herramientas de escritorio de .NET Framework 4.8 se puede compilar la solución de forma habitual.

Trazabilidad y alcance pendiente

Se usaron como referencia la consigna Proyecto_Analizador_Lexico_CSharp_WinForms.pdf (Entregable 3, páginas 10-11), la especificación léxica del grupo y el documento del Entregable 2. El repositorio del grupo es github.com/Zenite50/C-Extended. El [video de demostración en OneDrive](#) dura 3 minutos y 48 segundos y no se almacena en el repositorio.

Uso de IA: Codex asistió en la implementación, la elaboración de pruebas y la redacción de este informe. El grupo debe revisar, comprender y declarar este apoyo al realizar sus propios commits y la entrega, conforme a las normas del proyecto.

Siguiente etapa: conectar el núcleo probado con los controles Windows Forms y la exportación de resultados, actividades del Entregable 4.